Utilization of VAT for Hot-start of TSP Solutions

Scott Phillips (0009-0003-9169-7519) and Kelly Cohen PhD

[1]University of Cincinnati, Cincinnati, Ohio 45221
cohenky@ucmail.uc.edu
sphillips@nexigen.com

Abstract. Utilizing Visualization Assessment for Tendency (VAT) for acceleration of Ant Colony Optimization (ACO) Traveling Salesman Problem (TSP) solutions. The VAT, being based on a minimum spanning tree (MST) search, can be proven to provide at worst a 2x the optimal tour distance for a starting point. The testing was performed using sample datasets where a nearly optimal route could be computed analytically for comparison. The hot-start ACO achieved very good results compared to the analytical solution in much shorter time than the random-initialization ACO case. As a consequence of being an MST, VAT provides an excellent initial configuration (or "hot-start") for further refinement by local swarms, in addition to supporting recursive local cluster optimization. Further work needs to be performed to identify whether the converse is also true: ACO methods of creating the MST to approximate a VAT, thereby providing a trivially parallelizable method for cluster identification.

Keywords: VAT, TSP, Ant Colony Optimization

1 VAT Background

Visual Assessment for Tendency (VAT) is a cluster identification technique pioneered by Bezdek [1], [2]. It converts an *Nx2* matrix of observations into a *NxN* dissimilarity matrix **D.** While there are multiple dissimilarity measures that can be used, one of the most common is the Euclidean distance, or *L2-norm*. If the L2-norm is used, then the dissimilarity matrix becomes a symmetric distance matrix, and the observations become points, or "cities" on the 2D plane.

The VAT then uses a Minimum Spanning Tree (MST) approach to finding the optimal ordering of the columns of **D** to minimize differences from one row r_i to the next row r_{i+1}. It then generates the Reordered Dissimilarity Image (RDI), with the minimum distances aligned along the diagonal. When this **D** matrix is viewed as an image, "dark blocks" will appear that signify both cluster existence and cluster size.

The algorithm as proposed by Bezdek [1] et al is very similar to Prim's algorithm, which is used to identify the optimal tree routing through a collection of nodes, respecting which nodes and edges are connected.

2 ACO Background

Ant Colony Optimization (ACO) is a form of swarm optimization, wherein “ants” explore the possible search space, balancing the local optimal choice against the previously explored choices as expressed by a “pheromone trail”. The "pheromone trail” evaporates over time, and the most recent ants keep refreshing it. The strength of a given pheromone is inversely proportional to the cost function, so more optimal solutions have stronger pheromone trails.

While there are continuous and mixed-domain ACO techniques, one of the very common problems is a combinatorial optimization problem, the Traveling Salesman Problem (TSP). In short, the TSP is solving the problem of how short a path visits each and every city and returns to the start. The pheromone trail ends up being a 2D matrix **P** of the same shape as the Distance matrix **D**. As each ant traverses the cities, it picks a random city from those not visited, with the probability of each city being related to the immediate distance and the number of ants which made that choice before “pheromone trail”. When an ant has visited all cities and returned to the start, its pheromone trail is inversely related to the total distance it has traveled.

Because ACO is a stochastic optimization method, it does not require a cost function to be continuous or differentiable, only comparable. This relaxes a lot of common optimization constraints, but makes the problem susceptible to initialization issues. While ACO will not get stuck in (automatically find) the nearest local optima (unlike gradient-descent methods), it is not guaranteed to find an optima at all. As a result, most ACO problems are solved in finite time, a finite number of *generations*, and the optimal solution found is returned. As a result, it is helpful to have a reasonable initial guess of a solution, a so-called “hot start” to the ACO process. [5]

3 The Connection

If the dissimilarity matrix **D** is taken to be the distance matrix where the city numbers match the row/column numbers of the matrix (because it is symmetric when using the L2-norm), the VAT matrix **D`** is a permutation of the rows and columns that minimize the distances on the primary diagonals. A distance matrix will have *0* in the primary diagonal, since the Euclidean distance between a point *p* and itself is, by definition, zero.

That being said, the off-diagonal entries *(p,q)* represent the distance from a city *p* to city *q*. The total distance of the tour *T* is the sum of the distances from each city to the next, and then back to the start (if necessary). Because the graph is connected, closed, and symmetric, the total distance for a tour *T* does not change based upon the choice of starting city, only on the order of the cities in the tour. An intuitive tour would be the off-1 diagonal of **D`**, representing visiting each of the reordered cities sequentially. The return to start is the entry $\mathbf{D`}_{1,N}$, representing the distance from city *1* to city *N*. Since the VAT algorithm identifies the reordering, it is possible to identify the actual tour order *T* in the original city order, not just in the **D`** order.

This combination of distance and ordering provides a solution to the MST, as well as the corresponding TSP. In addition, because the TSP is, in the worst case, visiting every entry in the MST twice, the MST tour provides an upper bound on the best possible tour [4]:

$$Cost(tour_{best}) \leq 2 * Cost(MST) \tag{1}$$

3.1 Examples

The sample case used was designed to have obvious clusters as well as overall structure. This structure is a half-circle or a full circle with small, regular N-gons uniformly distributed around the primary circle (which itself is approximated as an N-gon). The variables in question are *N_clusters* and *N_cities_per_cluster*. By utilizing this structure, an accurate approximation to the optimal tour could be computed analytically by the following formula:

$$T_{optimal} = P_{polygon} + N_{cities} * P_{city_{polygon}} - N_{cities} * D_{city_{polygon}} \tag{2}$$

By keeping *D_polygon* > *D_city_polygon*, the optimal tour will encompass each small cluster in sequence around the perimeter. A random tour will crisscross the center, leading to a much longer (and therefore worse-performing) tours.

The results shown in Table 1 were all performed on an Intel Core i7-1135 with 16GB of RAM using 64-bit Python. The VAT method was provided by an optimized Python library, while the ACO method was implemented manually by the authors.

The approximately optimal value is provided for comparison, but the time it takes is not relevant. Since it is the reference distance, the *% Change* in distance is, by definition, 100%. *Random* is simply the tour distance for a random permutation of the cities. It too is provided for reference on how long an unoptimized tour can be. By default, a random initialization ACO method starts with this distance and refines it down to the distances shown below. As is clearly evident, the HS-ACO (hot-start ACO) performs better than the regular (random) ACO, while taking slightly longer due to the initialization time of the VAT.

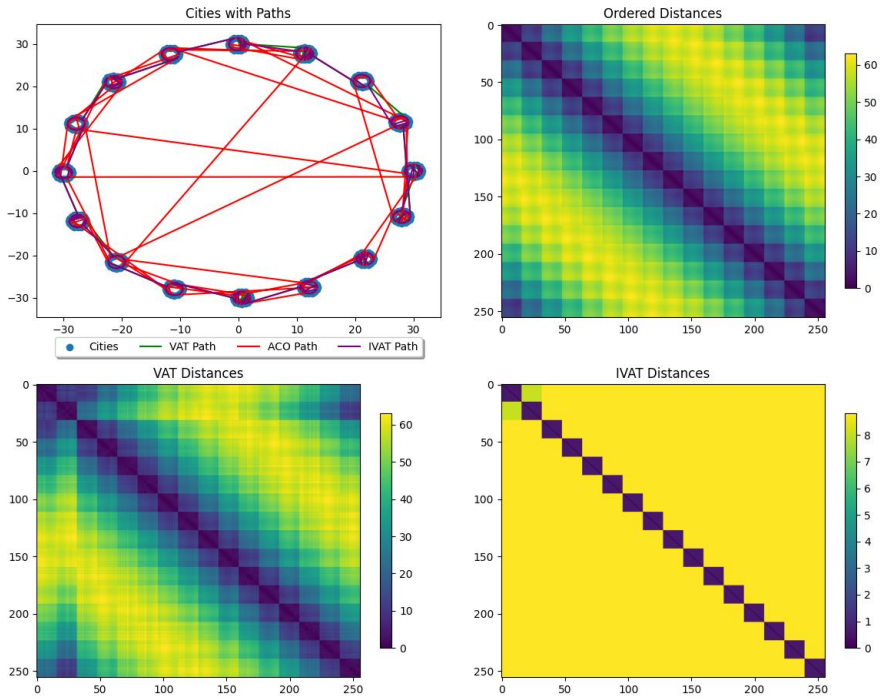
IVAT, the Improved Visualization Assessment Tendency for clustering, method beats the optimal distance because it does *not preserve information*. For other sample layouts, depending on *N_clusters* and *N_cities_per_cluster*, the computed "distance" can be lower than the optimal distance. It is shown for example, but should not be considered a valid method to hot-start the ACO, since it effectively drops some of the cities to make the actual clusters higher contrast when visualized.

Table 1. Method, Time, Distance Comparison

Method	Time [s]	Distance	%Change
Approx Optimal	0.000	289	100%
Random	0.000	10,074	3484%
VAT	0.345	408	141%
IVAT	0.345	281	97.1%
HS-ACO	4.95	408	141%
ACO	4.10	1592	551%

As is visible in Fig 1, the HS-ACO method finds a route around the cities of each cluster in nearly optimal order. In different runs, it finds the optimal order without back-tracking. The random-start ACO still requires more time to remove the random center crossings which contribute to the increased length (and reduced tour performance).

Fig. 1. City Paths and VAT/IVAT comparison



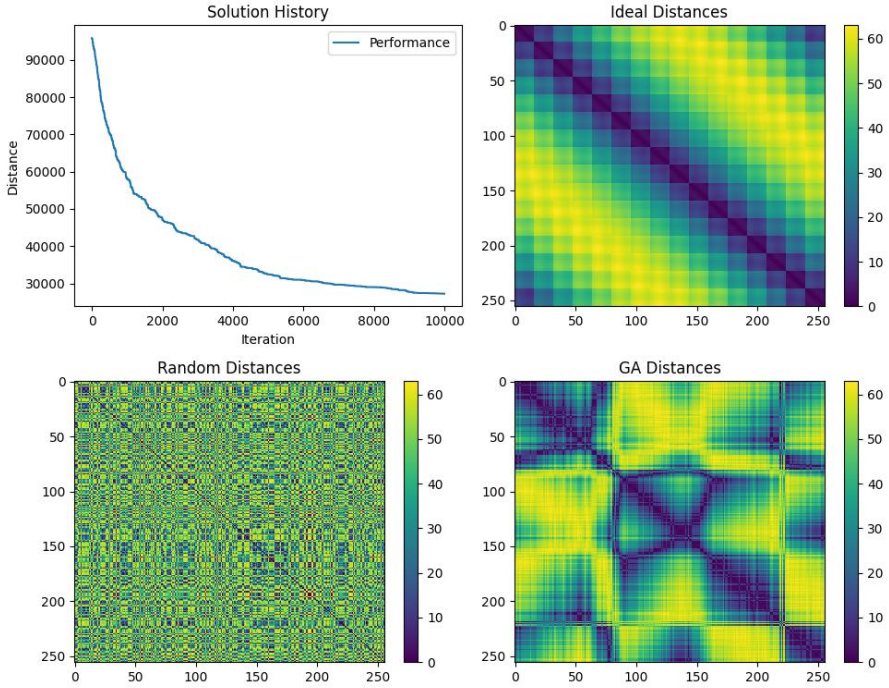
4 The (other) Connection

A natural follow-up question to "can the VAT hot-start a TSP?" is "can a TSP solution help identify the VAT?" To explore this case, we took a "random" permutation of cities based upon the example design of 3.1. This provides confidence that the optimal VAT clustering is very distinct and structured, while appearing random on first glance. The only constraint applied to the permutation was that city-1 (the start) remained at column-1, thereby ensuring the ACO started on a consistent city in every run. All rows and columns were permuted the same way, thereby keeping the permuted distance matrix **DP** symmetric.

At first, a naïve ACO TSP solver from 2 was used. It was found that the ACO solution did not converge quickly relative to the size of **DP**. Since the final outcome should always be the optimal one, a random permutation system was applied. For this method, the initial cities were assumed to be in order $co=\{1,\ldots N\}$, and random pairs of cities were swapped to produce order *cp*. The tour length was then computed directly (an O(N) operation). If the $Tour(cp) < Tour(co)$, then the permuted order is assigned to *co*. Otherwise, the permutation is undone and another pair of random cities is permuted. This operation is repeated for a predefined number of times *N_permutations*. The final permuted order is then returned, and the approximate VAT matrix **aD** is visualized using the normal VAT techniques.

4.1 Example

Fig. 2. TSP Approximation of VAT



As is visible in Fig 2, the principle diagonal blocks become stronger. In addition, a set of squares along the principle diagonal is starting to appear. Because of the nondeterministic nature of random permutation, additional gains become asymptotically more rare and therefore computationally expensive. As result, there is a need to further optimize the pairwise selection method as the matrix is increasingly ordered. Further research is required to determine if this permutation method can be effectively parallelized, thereby providing scaling advantages over the single-threaded VAT approach.

Because the VAT is a MST over the dissimilarity matrix, ACO methods of finding an optimal MST should be applicable to this technique. There already exist several methods for constructing an approximately optimal connection graph using ACO techniques [6]. Because of the previously identified connection between the dissimilarity matrix **D** and the minimum spanning tree of the VAT, it should be straightforward to apply some of the existing techniques for ACO generation of MST's to compute a VAT. Because ACO methods are trivially parallelizable, unlike Prim's algorithm, there should be much better results for larger matrix sizes. Currently, the VAT is limited to matrices of 1000x1000 or less [1] using aforementioned Prim's algorithm due to its N^3 time complexity.

While outside the scope of this paper, and a note for future work, analysis of the larger block structures that emerge could lead to a recursive, or block-based optimization method. This would enable local region parallel processing, as well as larger-

scale efficient ordering beyond what individual row-column permutation would provide.

5 Conclusions and Suggested Future Work

Utilizing the VAT technique provides an excellent initial guess to solving TSP type problems with ACO. The results indicated much better tour distances than a random-initialization ACO result would achieve in a similar amount of time. In addition to providing an upper bound on the optimal TSP tour length, it provides local cluster information which can lead to targeted local optimization. Further research needs to be performed on how the IVAT process can be used to provide a further tour length local optimization.

The converse, usage of permutation methods on a distance matrix to approximate a VAT did not prove as useful. The convergence rate was extremely slow, and the output was not optimal for identifying clusters. Further research will need to be performed to identify better local and regional optimization methods, as well as parallelization approaches to create a more competitive solution. Future work also entails developing the connection between ACO MST methods and proper VAT generation, since this would be applicable to much larger systems and trivially parallelizable with existing techniques.

References

1. Wang, L., Nguyen, U.T.V., Bezdek, J.C., Leckie, C.A., Ramamohanarao, K. (2010). iVAT and aVAT: Enhanced Visual Analysis for Cluster Tendency Assessment. In: Zaki, M.J., Yu, J.X., Ravindran, B., Pudi, V. (eds) Advances in Knowledge Discovery and Data Mining. PAKDD 2010. Lecture Notes in Computer Science(), vol 6118. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-13657-3_5
2. Rathore, P. Bezdek, J., Santi, P., Ratti, C., ConiVAT: Cluster Tendency Assessment and Clustering with Partial Background Knowledge, https://arxiv.org/pdf/2008.09570v1
3. Havens, Timothy & Bezdek, James. (2012). An Efficient Formulation of the Improved Visual Assessment of Cluster Tendency (iVAT) Algorithm. Knowledge and Data Engineering, IEEE Transactions on. 24. 1 - 1. 10.1109/TKDE.2011.33.
4. Chen, Y. MST-Approximation Implementation of TSP Problem, https://realcyh.github.io/yuhang-chen/tsp/, last accessed 2025/04/15
5. Neroni, M. Any Colony Optimization with Warm-Up, Algorithms 2021; 14, 295, https://www.mdpi.com/1999-4893/14/10/295, last access 2025/06/14
6. Neumann, F., Witt, C., Ant Colony Optimization and the minimum spanning tree problem. Elsevier Theoretical Computer Science Volume 411, Issue 25, Pages 2406-2413, https://doi.org/10.1016/j.tcs.2010.02.012 , last accessed 2025/06/14